HANGZHOU DIANZI UNIVERSITY

Information Security Laboratory

LABORATORY WORK No. 1

Secure REST API Development with CI/CD and Real CVE Vulnerability Analysis

Variant	N/A (not assigned)
Student	李泽涛 / LiZetao
HDU number	23320332
Group	23320113
Repository	https://github.com/mycbdys/lab1
Submission date	19 October 2026

Python 3.13 / Flask 3.1 | SQLite | pytest | Bandit | pip-audit | Node.js 20 / EJS

1. Summary

This work implements a small secure REST API in Python/Flask, runs automated SAST and SCA checks in GitHub Actions, and analyses a real vulnerability: CVE-2022-29078, a remote-code-execution flaw in the EJS template engine. The API has three endpoints, bcrypt password storage, JWT authentication, parameterised SQL and output encoding. The CVE was reproduced in a controlled environment, its root cause was identified in the EJS source, and the fix (EJS 3.1.7 plus safer application code) was verified.

Repository: https://github.com/mycbdys/lab1 | CI: https://github.com/mycbdys/lab1/actions/runs/39195640683

2. API and security controls

Endpoint	Auth	Purpose
POST /auth/login	No	Verify credentials, return a 15-minute JWT
GET /api/data	Bearer JWT	Return only the caller's posts, HTML-escaped
POST /api/posts	Bearer JWT	Create a post owned by the authenticated user

SQL injection is prevented with bound parameters for every query; user input is never concatenated into SQL. Stored XSS is prevented by HTML-escaping all text on output and by a strict Content-Security-Policy. Passwords are hashed with bcrypt (cost 12, unique salt); login errors are identical for unknown users and wrong passwords, and five failed attempts per minute return 429. JWTs are HS256 with a pinned algorithm, issuer, audience and required exp/iat/sub claims, so tampered, expired and alg=none tokens are rejected. Every data query is scoped by the user id from the verified token, and responses carry nosniff, DENY frame options, no-referrer and no-store headers.

3. CI/CD and results

Job	Tool	Gate
Unit tests	pytest 9.1.1	21 tests must pass
SAST	Bandit 1.9.4	No medium/high findings
SCA	pip-audit 2.10.1	No known vulnerable dependency

The workflow runs on every push and pull request. Results: 21/21 tests passed, Bandit reported 0 issues in 303 lines of code, and pip-audit found 0 known vulnerabilities in 44 packages. The same pipeline passed on GitHub's clean runner, so the result is not only local.

4. CVE-2022-29078

Field	Value
CVE	CVE-2022-29078 (GHSA-phwq-j96m-2c2q)
Component	EJS template engine for Node.js
Affected / fixed	EJS < 3.1.7 / EJS 3.1.7
Class / severity	CWE-74 injection leading to RCE / CVSS 9.8 Critical

The vulnerable app forwards the whole query object into res.render(). EJS merges data.settings['view options'] into its compiler options, and outputFunctionName is concatenated directly into the generated JavaScript. A crafted request therefore executes a command while the page still returns 200.

```
GET /?settings[view options][outputFunctionName]=
x;process.mainModule.require("child_process").execSync("id > pwned.txt");s
```

Observed in Docker (GitHub Actions): HTTP 200 with the normal page, and pwned.txt inside the vulnerable container contained uid=0(root). The same request against the fixed app (EJS 3.1.7, scalar-only render data) returned the normal page and created no file; the library also rejected the option as 'not a valid JS identifier'.

Root cause and fix

Three conditions combine: (1) the application passes attacker-controlled data as render options; (2) EJS copies settings['view options'] into internal compiler options without an allow-list; (3) outputFunctionName is inserted into

generated source, so the payload becomes executable statements. The fix is applied at both layers: upgrade to EJS 3.1.7, which validates the option as a JavaScript identifier, and stop passing the query object, passing only a 64-character scalar name. Neither fix relies on a blocklist of characters.

5. Evidence (screenshots)

The screenshots below were captured from the public GitHub repository and Actions pages; they are not mock-ups. The first shows the three CI jobs passing, and the second shows the Docker run steps plus the files produced inside the containers.

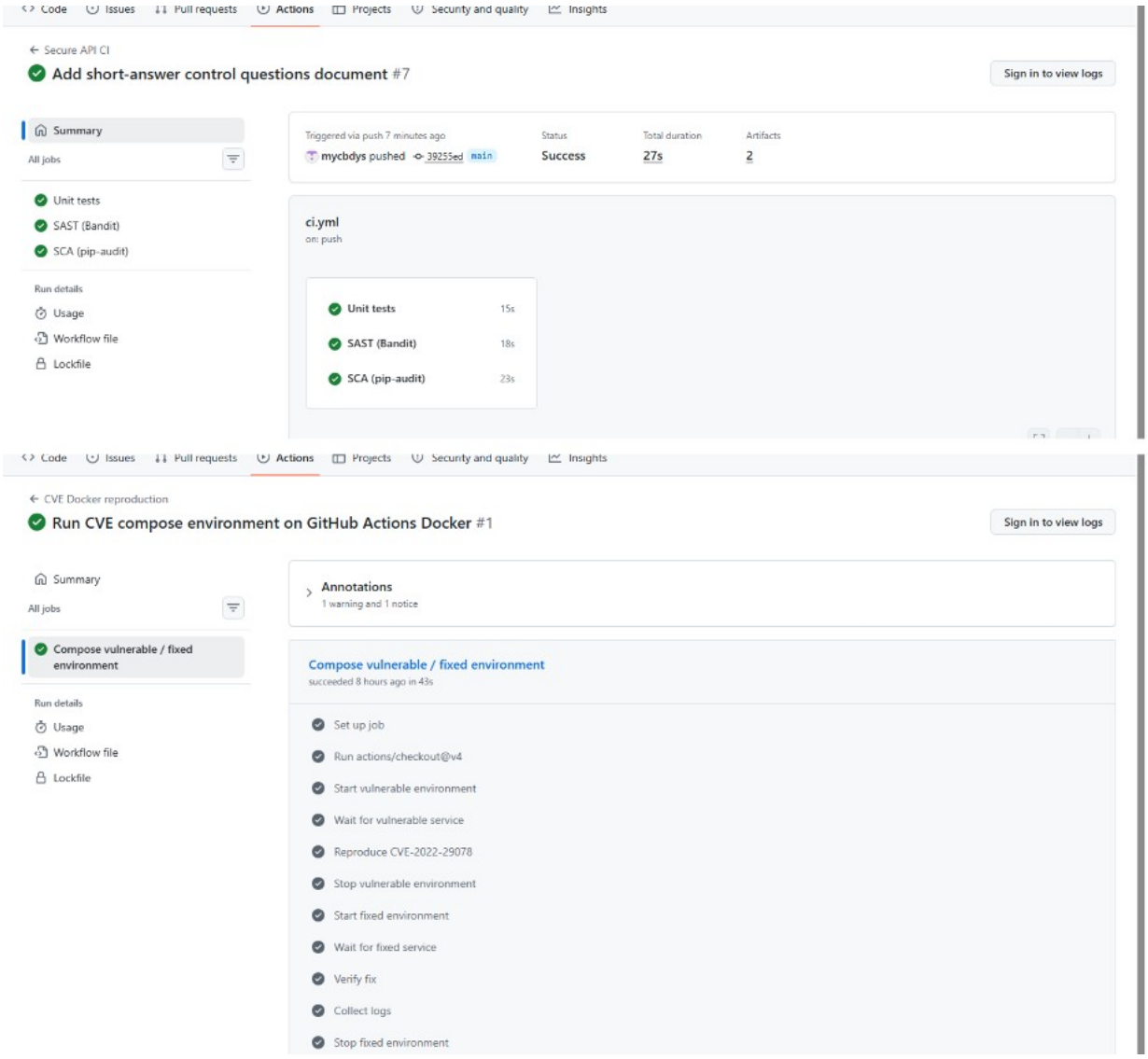


Figure 1. GitHub Actions: Unit tests, SAST (Bandit) and SCA (pip-audit) all pass; the Docker job builds the vulnerable stack, reproduces the CVE, starts the fixed stack and verifies it.

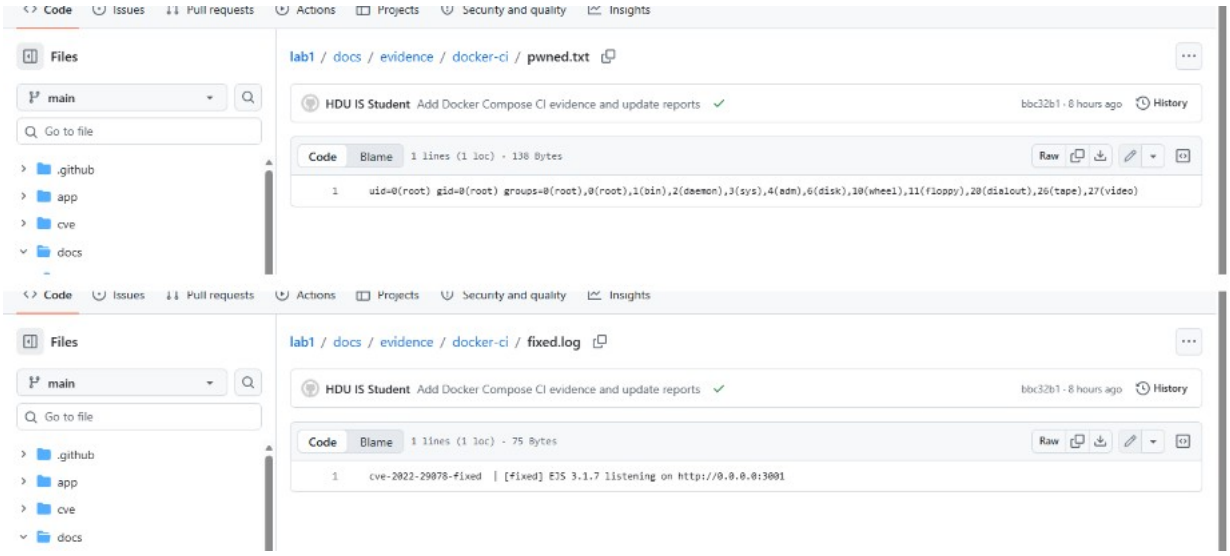


Figure 2. Docker evidence: pwned.txt from the vulnerable container contains uid=0(root); the fixed container runs EJS 3.1.7.

6. Testing and submission

The pytest suite covers login, SQL injection attempts, bcrypt storage, rate limiting, missing/tampered/expired/alg=none tokens, per-user isolation, XSS escaping, SQL metacharacters stored literally, and security headers. Manual cURL evidence is in evidence/api-demo.txt; CVE evidence is in cve/CVE-2022-29078/evidence/ and docs/evidence/docker-ci/.

Deliverables: public repository https://github.com/mycbdys/lab1; pipeline https://github.com/mycbdys/lab1/actions/runs/39195640683; control questions in the separate document Lab1_Control_Questions.docx. The fixes are under cve/CVE-2022-29078/fixed/ with patches/fix.diff.

Limitation: Docker Desktop is not installed on the local Windows host (it needs administrator rights, WSL2 and a reboot). The same compose files were executed in real Docker on the GitHub Actions runner (run 39195350651): the vulnerable container ran id as root and the fixed container blocked the attack. The local run used Node.js 20 with the same package versions.